The University of Hong Kong

Faculty of Engineering
Department of Computer Science

COMP3230 Principles of Operating Systems

Date: December 13, 2023 Time: 2:30 pm - 5:30 pm

Only approved calculators as announced by the Examinations Secretary can be used in this examination. It is the candidates' responsibility to ensure that their calculator operates satisfactorily, and candidates must record the brand and type of the calculator used in the space provided.

This is an open book examination. Candidates may bring to their examination any printed/written materials.

Total 100 points.

Answer ALL questions.

Answer each question in the space provided. If you need to continue an answer onto the back of a page, clearly indicate that and label the continuation with the question number.

Question No.	Mark
1	/10
2	/10
3	/20
4	/15
5	/10
6	/20
7	/15
Total	**100**

Question 1 – Multiple-Choice Questions (10 points)

Two marks for each question. Each question has **ONLY ONE** correct answer. You can OPTIONALLY write down your assumptions very briefly, if any.

(a) Which of the following events will cause the system to resume execution from the same instruction that triggered the event?

A. Completing a system call.
B. Handling a division-by-zero exception.
C. TLB miss.
D. Handling a timer interrupt.

Your Answer:

(b) A user application performed an illegal memory access, which causes an exception and terminates the process. Considering the scenario, which of the following statements is true?

A. The CPU directly terminates the application.
B. The CPU must be in the kernel mode right before this exception occurs.
C. The OS is the first to detect such an exception.
D. The OS handles this exception with support from the CPU hardware.

Your Answer:

(c) Which of the following is NOT a memory management technique used in operating systems?

A. Paging
B. Segmentation
C. Caching
D. Fragmentation

Your Answer:

(d) Multiple bits are maintained in a page table entry. Which of the following bits are most useful for page replacement?

A. Dirty bit
B. Valid bit
C. Reference bit
D. Present bit

Your Answer:

(e) Consider the following code fragment:

```
if (fork() == 0)
{ a = a + 5; printf("%d, %d\n", a, &a); }
else
{ a = a - 5; printf("%d, %d\n", a, &a); }
```

Let u, v be the values printed by the parent process, and x, y be the values printed by the child process. Which one of the following is TRUE?

A. u = x + 10 and v = y
B. u = x + 10 and v != y
C. u = x + 5 and v = y
D. u = x + 5 and v != y

Your Answer:

Question 2: Concurrency (10 points)

Are the following statements **TRUE** or **FALSE**? Justify your answer.
[Note: 2 points for each question. No points will be given without explanation.]

(a) A working solution to a concurrency problem on a multiple core system should also be applicable to a single core system; however, the reverse is not necessarily true.

(b) In the readers-writers problem, threads X and Y are allowed to simultaneously access the shared resource if and only if either X and Y are both reading or X and Y are both writing.

(c) If a thread executes a sem_wait() call on a binary semaphore, it must eventually execute a sem_post() call on the same semaphore. Failure to do so will result in a synchronization bug.

(d) If thread X calls cond_signal() before thread Y calls cond_wait(), it allows thread Y to return from cond_wait() without blocking.

(e) If thread X calls sem_post() before thread Y calls sem_wait(), it allows thread Y to return from sem_wait() without blocking.

Question 3 – Page Replacement (20 points)

Consider a computer system with a main memory consisting of 4 frames and a page table for tracking the pages in the memory. The following sequence of page references is given:

3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 8, 9, 7, 9, 3, 2, 3, 8, 4.

Assume that initially, the memory is empty. You are required to simulate the page replacement process and determine the number of page faults that occur using the following page replacement algorithm.

If there is a tie in the selection process, you can assume that the oldest page will be replaced in order to break the tie. Solve the problem by showing the sequence of page faults and the pages present in the memory after each reference.

(a) (10 points) Using the Least Frequently Used (LFU) algorithm.

	3	1	4	1	5	9	2	6	5	3	5	8	9	7	9	3	2	3	8	4
F1																				
F2																				
F3																				
F4																				

(b) (10 points) Using the Clock Replacement algorithm.

	3	1	4	1	5	9	2	6	5	3	5	8	9	7	9	3	2	3	8	4
F1																				
F2																				
F3																				
F4																				

Question 4 – Multi-level Paging (15 points)

Consider a processor that supports 48-bit virtual address and 40-bit physical address. To manage virtual memory, the operating system employs a multi-level paging scheme. For each page or directory table, the system uses a 32-bit entry to store the page frame number and 8 bits of control information (e.g., present bit, valid bit, etc., which use 8 bits in total). The system adopts a scheme that minimizes the number of memory frames for storing the directory and page tables.

(a) (10 points) Determine how many levels of tables are required. Find the structure of the processor's virtual address, i.e., how many bits for the 1st level page table, 2nd level page directory, . . . etc.?

(b) (5 points) If the system uses a TLB to cache frequently used address mapping, how big would a TLB entry be for this system? Justify your answer.

Question 5 – Synchronization Control (10 points)

Below is an implementation for the Producer and Consumer Problem, where we assume that a Producer/Consumer thread only produces/consumes one item each time.

```
buffer[sizeofbuffer];
mutex_t mv;
cond_t notFULL, notEMPTY;
count = 0;
```

```
Producer( ){
while (1)
{
  lock(mv);
  while (count == sizeofbuffer)
    cond_wait(notFULL, mv);

  <<< put item into the buffer >>>

  cond_signal(notEMPTY);
  unlock(mv);
  // remaining work
}
}
```

```
Consumer( ) {
while (1)
{
  lock(mv);
  while (count == 0)
    cond_wait(notEMPTY, mv);

  <<< get item into the buffer >>>

  cond_signal(notFULL);
  unlock(mv);
  // remaining work
}
}
```

Revise the code to implement a version where a Producer/Consumer thread will produce/consume *k* items each time. Note that *k* is different for different threads. Assume k <= sizeofbuffer.

```
// declarations of additional variables besides those defined above
```

```
Producer () {
while(1) {
```

```
Consumer () {
while(1) {
```

Question 6 – Deadlock Avoidance (20 points)

Suppose there are three processes (P1, P2, P3) and three resource types (R1, R2, R3) in a system that avoids deadlock using the Banker's algorithm. The maximum resource requirement matrix and the current allocation of resources are given as follows, respectively:

Max =

	R1	R2	R3
P1	4	4	2
P2	4	3	1
P3	13	5	3

Loan =

	R1	R2	R3
P1	4	0	0
P2	1	2	1
P3	2	1	0

The total amount of each resource type in the system is given by the vector [13, 6, 4].

(a) (5 points) Is the current state safe or not? Justify your answer.

(b) (5 points) Suppose P3 requests for 6 units of R1. Will this request be granted or refused? Justify your answer.

(c) (10 points) While the original Banker's algorithm assumes a fixed number of processes, we can still apply it in case of limited changes. Let us consider a new process, P4, which arrives with a maximum resource claim of [4, 2, 3] for the three types of resources. Now suppose P4 requests 3 units of R3. Should the request be granted or refused? Justify your answer. (This question is independent of (b)).

Question 7 – File Systems (15 points)

Consider a USB flash drive device with a storage capacity of 64 GiB that uses the tabular allocation scheme to manage its storage space. In this case, let's assume that the system has a disk block size of 4 KiB, and each entry in the block allocation table stores a 32-bit disk block pointer. [Note: 1 KiB = 2^{10} bytes and 1 GiB = 2^{30} bytes.]

(a) (6 points) How many disk blocks on the flash drive are required to store the block allocation table?

(b) (4 points) Suppose the file system uses a data block bitmap for free space management. How many disk blocks are required to store the data block bitmap?

(c) (5 points) Consider a program that has opened a file containing 200 blocks of data. Assume the block allocation table, data block bitmap and the file's file control block are already loaded to the memory. How many disks I/O operations (read and write) will result when the system removes the 99th block from the file? Assume that after the system updates the metadata, it pushes the update to the disk.

[THIS PAGE INTENTIONALLY LEFT BLANK]

***** END OF PAPER *****